Virtual Worlds

Programming Assignment #1 Report



학과/학부	소프트웨어학부
학번	2021203015
이름	강선후

2025. 4. 12

■ Fulfill table

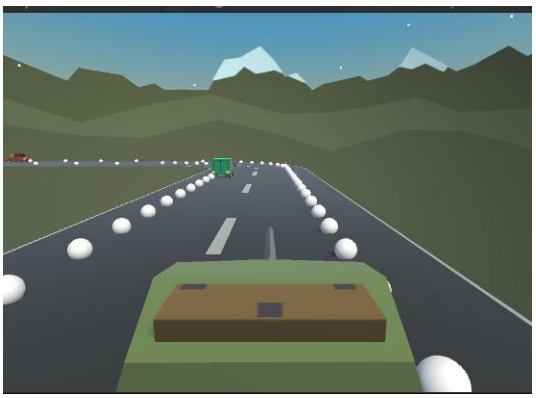
Easy 1	Camera switcher	O
Easy 2	Revival	O
Medium 1	Oncoming vehicles	O
Medium 2	Road layout	O
Hard	Make a curved road mesh	O
Expert	Oncoming vehicle on curved road	O

■ For each requirement, explain how you fulfilled it

☐ Easy 1: Camera Switcher

View1, View2 오브젝트를 생성하여 적절한 위치와 각도로 배치한 뒤, 다음과 같이 shift 버튼 KeyDown 시 시점을 각 View로 스위치하게 하였습니다.

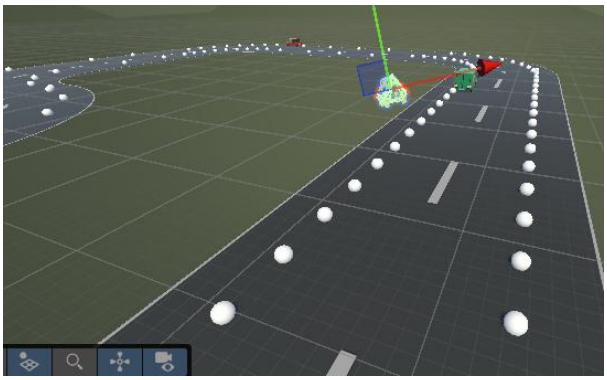
```
    void Update()
    {
        if (Input.GetKeyDown(KeyCode.LeftShift) || Input.GetKeyDown(KeyCode.RightShift))
        {
            currentViewIndex = (currentViewIndex + 1) % views.Length;
        }
        transform.position = views[currentViewIndex].position;
        transform.rotation = views[currentViewIndex].rotation;
    }
}
```

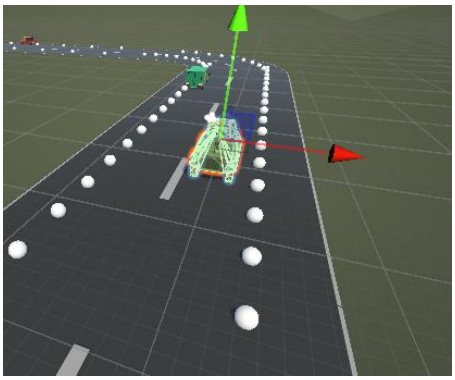




☐ Easy 2: Revival

fallThreshold를 설정하여, 특정 높이 이하로 떨어지면 respawnPosition으로 옮겨주고, position/rotation/linearVelocity/angularVelocity를 전부 초기화해주었습니다.

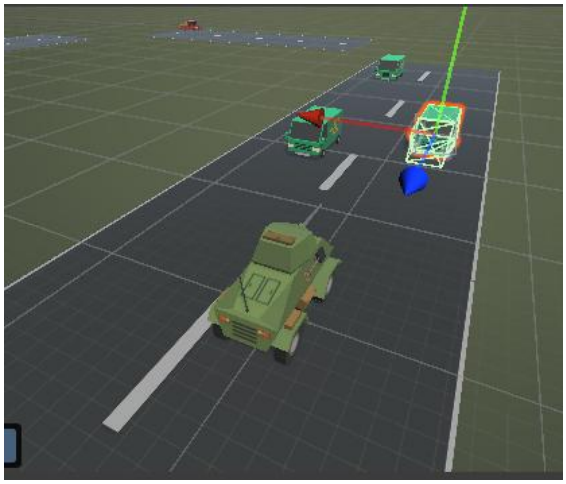


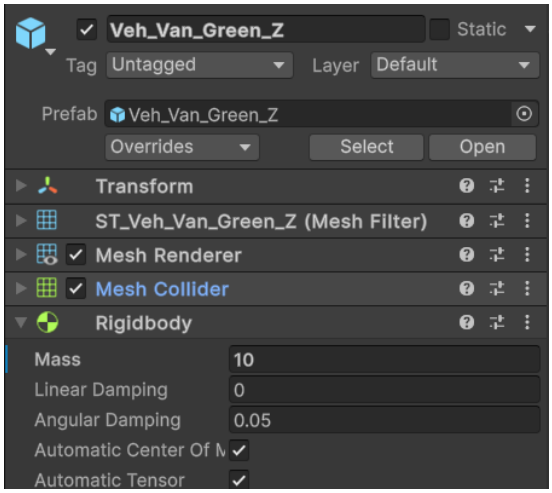


☐ Medium 1: Oncoming Vehicles

차량 이동 구현 : Create-With-Code : Prototype2의 알고리즘 적용

차량의 rotation을 조정하여 player와 마주보게 배치, play 시 moveForward script에 따라 달려오게 구현. Component : Rigidbody를 추가하여 피하지 않으면 충돌하게 설정하였습니다.





☐ Medium 2: Road Layout



☐ Hard: Make a Curved Road Mesh

1. SetVertices

```
float angleStep = curveAngle / dense;
Vector3 pivot = new Vector3(radius, 0, 0);
```

회전 중심점 설정 : radius 값을 기준으로 위치를 계산합니다.

각도에 따라 회전한 후, 도로의 왼쪽과 오른쪽 끝의 점들을 계산해 vertices 리스트에 추가합니다.

2. SetTriangles / SetUV

메쉬를 구성하기 위해, 각 정점들을 삼각형으로 연결하여 면을 만들고, UV 맵핑을 설정해줍니다.

```
protected override void SetTriangles()
{
    for (int i = 0; i < vertices.Count - 2; i += 2
    {
        triangles.Add(i);
        triangles.Add(i + 2);
        triangles.Add(i + 1);

        triangles.Add(i + 1);
        triangles.Add(i + 2);
        triangles.Add(i + 3);
    }
}
```

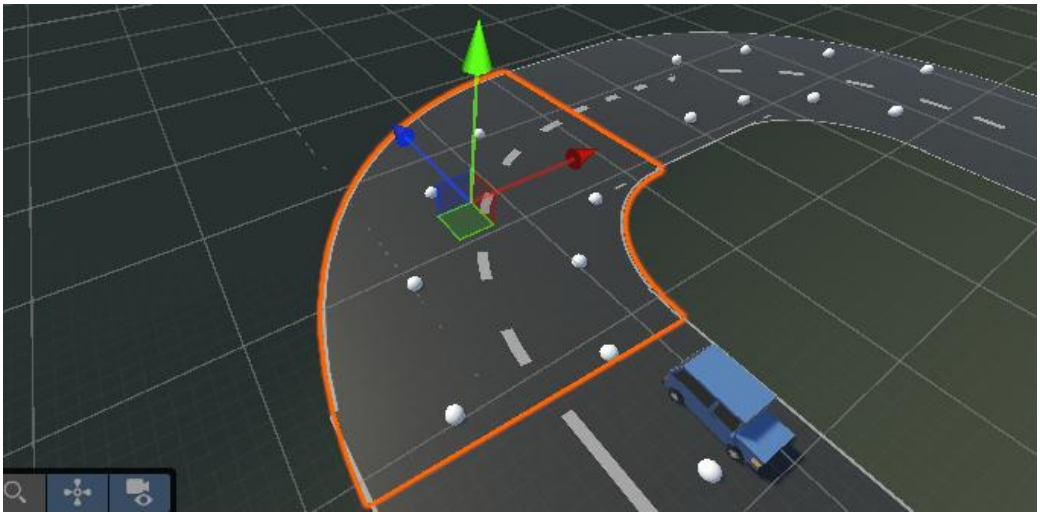
```
protected override void SetUV()
{
    uv.Clear();

    float angleRad = Mathf.Deg2Rad * curveAngle;
    float arcLength = radius * angleRad;

    for (int i = 0; i <= dense; i++)
    {
        float u = (arcLength / dense) * i / arcLength;
        uv.Add(new Vector2(u, 0));
        uv.Add(new Vector2(u, 1));
    }
}
```

3. SetNormals

도로가 위를 향하게 법선벡터(Vector3.up)까지 설정해주었습니다.



☐ Expert: Oncoming Behicle on Curved Road

앞서 만든 Road Layout을 따라 자동으로 움직이는 차들을 추가해주었습니다.

1. 각 도로(Straight/RightCurve/LeftCurve)에 맞는 WaypointGenerator 구현

2. 생성된 Waypoint를 차들이 따라갈 수 있도록 WaypointFollwer 구현

이렇게 2단계로 나누어 진행하였습니다.

1. WaypointGenerator

도로의 종류는 총 3개입니다. Straight Road/RightCurve Road/LeftCurve Road 각각은 다른 형태를 지니기에 Waypointer 생성에 대한 설정을 다르게 해주었습니다. Enum 형식으로 RoadType을 구분하고, switch문으로 case를 나누어 해당 도로에 맞는 Waypoint를 자동으로 생성할 수 있도록 해주었습니다.

```
private void GenerateStraightWaypoints()
{
    Vector3 origin = transform.position;
    Vector3 forward = transform.forward;

    for (int i = 1; i < dense; i++)
    {
        float t = (float)i / (dense - 1);
        Vector3 basePos = origin + forward * straightLength * t;

        CreateWaypoint(basePos - transform.right * laneOffset, leftLaneParent);
        CreateWaypoint(basePos + transform.right * laneOffset, rightLaneParent);
    }
}
```

2차선 도로임을 고려하여 양쪽 차선에 한번에 웨이포인트가 모두 생성되도록 구현하였습니다.

직선 도로에 비해 곡선 도로는 생각할 것이 조금 더 많았습니다. 우선 회전 중심점을 설정하고, 곡선 전체 각도를 dense 간격으로 나누어, Waypoint가 균일한 간격으로 배치되도록 하였습니다. 삼각함수를 이용해 원 위의 점 (x, z)를 구하고, 이를 center 기준으로 회전시켜 실제 월드 좌표에 맞는 위치로 설정하였습니다.

```
Vector3 right = Vector3.Cross(Vector3.up, (basePos - center).normalized);
CreateWaypoint(basePos - right * laneOffset, leftLaneParent);
CreateWaypoint(basePos + right * laneOffset, rightLaneParent);
```

center → basePos 방향의 오른쪽 벡터를 계산하여 해당 벡터를 기준으로 좌측 차선과 우측 차선에 각각 웨이포인트를 생성할 수 있도록 해주고, 각 웨이포인트는 leftLaneParent, rightLaneParent 오브젝트의 자식으로 관리하였습니다.



2. WaypointFollower

이제 차들이 생성된 Waypoints를 따라 이동할 차례입니다. 앞서 차선을 2개로 나누고 LeftLane/RightLane에 각각 Waypoint를 생성하였으므로, WaypointFollower도 차선을 구분하여 차량들이 움직일 수 있도록 Left/Right를 나누어주었습니다. 각 차량은 해당 차선(Lane)의 태그(Left/Right)를 기반으로 가장 가까운 Waypoint를 탐색하고, 순차적으로 도로를 따라 이동하도록 구현하였습니다.

```
int FindClosestWaypointIndex()
{
    float minDist = Mathf.Infinity;
    int closest = -1;

    for (int i = 0; i < waypoints.Count; i++)
    {
        float dist = Vector3.Distance(transform.position, waypoints[i].position);
        if (dist < minDist)
        {
            minDist = dist;
            closest = i;
        }
    }

    return closest;
}
```

차량은 목표 웨이포인트에 도달하면 진행 방향에 있는 가장 적절한 Waypoint를 탐색하고, 다시 Waypoint Set을 따라 움직이게 해주었습니다.

```
Vector3 toWaypoint = waypoints[i].position - transform.position;
float dist = toWaypoint.magnitude;

// 진행 방향과 유사한 방향인지 확인
float dot = Vector3.Dot(transform.forward, toWaypoint.normalized);
if (dot > 0.5f && dist < minDist)
{
    minDist = dist;
    nextIndex = i;
}
```

■ Conclude with some comments on your work

☐ Key challenges you have successfully tackled

차량이 Waypoint를 따라 자동으로 곡선 도로를 주행해야하는 Expert 난이도의 과제를 제외한 나머지는 전부 성공적으로 구현해낸 것 같습니다. 다만, 곡선 도로를 만드는 과제에서 계속해서 예상했던 모양으로 도로가 생성되지 않아 고민했던 시간이 길었던 것 같습니다.

☐ Limitations you hope to address in the future

현재는 2차선 도로임을 고려하여 한 번의 실행으로 양쪽 차선에 웨이포인트가 모두 생성되도록 구현하였지만, WaypointGenerator를 각각 배치해야 하는 구조로 인해 오히려 배치 과정이 번거로울 수 있음을 느꼈습니다. 다음에는 도로 생성과 동시에 해당 도로에 맞는 Waypoint도 자동으로 생성되도록 통합하는 방식이 더 효율적일 수 있겠다는 생각을 하였습니다.

■ Add a link to a video that runs the program (e.g. YouTube)

https://www.youtube.com/watch?v=UPX1eyO1rqA

■ Additional Note

업로드한 unitiyPackage File을 import하여 실행해보니, 이동 입력(wasd)이 되지 않는 것을 확인하였습니다.

Edit > Project Settings > Player > Active Input Handling > Old / Both 둘 중 하나로 설정을 하여 자동으로 프로젝트가 재시작된 후에는 정상적으로 동작 가능하니 참고해주시면 감사하겠습니다.

Editor Version : 6000.0.42f1